

28. Introduction to JavaScript

What is JavaScript

- JavaScript (JS) is a programming language used to make websites interactive.
- Adds functionality and behavior to web pages.

Uses

- Button actions
 - Form validation
 - Dynamic content
 - Games
 - API calls
 - Web applications
-

How JS Works

HTML → Structure

CSS → Styling

JS → Functionality

- Browser loads HTML and CSS first.
 - Then JavaScript executes.
-

Interpreted Language

- JavaScript is an interpreted language.
- Runs line by line.
- No separate compilation required.

```
console.log("Hello");
```

JS Engine

- JS Engine executes JavaScript code.
- Converts JS into machine code.

Popular Engines

- Chrome → V8
- Firefox → SpiderMonkey

Steps

1. Read code
 2. Convert code
 3. Execute code
-

Linking JS with HTML

1. Inline JS

```
<button onclick="alert('Hi')">
```

Click

```
</button>
```

2. Internal JS

```
<script>
```

```
  console.log("Hello");
```

```
</script>
```

3. External JS

HTML

```
<script src="app.js"></script>
```

app.js

```
console.log("Hello");
```

- External JS is best practice.
-

Script Placement

- Place `<script>` before `</body>`.
- HTML loads first, then JS runs.

```
<body>
```

```
  <script src="app.js"></script>
```

```
</body>
```

console.log()

- Prints output in console.

```
console.log("Hello JS");
```

alert()

- Shows popup message.

```
alert("Welcome");
```

29. Variables & Data Types

Variables

- Used to store data.

```
let name = "Aditya";
```

var

- Old keyword
- Re-declare allowed
- Function scoped

```
var age = 20;
```

let

- Modern keyword
- Update allowed
- Re-declare not allowed
- Block scoped

```
let age = 20;
```

const

- Fixed value
- Cannot update
- Block scoped

```
const pi = 3.14;
```

Primitive Data Types

- Single values

Types

- String
- Number
- Boolean
- Undefined
- Null
- BigInt
- Symbol

```
let a = "Hello";
```

```
let b = 10;
```

```
let c = true;
```

Non-Primitive Data Types

- Collection of values

Types

- Object
- Array
- Function

```
let arr = [1,2,3];
```

typeof

- Checks data type

```
typeof "Hello"
```

```
string
```

```
typeof 10
```

```
number
```

30. Operators

Arithmetic Operators

- Used for calculations

Operator Meaning

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus
**	Power

```
let a = 10;
```

```
let b = 2;
```

```
console.log(a + b);
```

Assignment Operators

- Assign values

Operator Example

```
=      a = 5
```

```
+=     a += 2
```

```
-=     a -= 2
```

```
*=     a *= 2
```

```
/=     a /= 2
```

```
let a = 5;
```

```
a += 2;
```

Comparison Operators

- Compare values
- Return true or false

Operator Meaning

```
==     Equal
```

Operator Meaning

===	Strict Equal
!=	Not Equal
!==	Strict Not Equal
>	Greater
<	Smaller
>=	Greater Equal
<=	Smaller Equal

5 == "5"

5 === "5"

Logical Operators

Operator Meaning

&&	AND
	OR
!	NOT

true && false

Ternary Operator

- Short form of if-else

let age = 18;

age >= 18 ? "Adult" : "Minor";

Increment / Decrement

Increment ++

- Increases value by 1

let a = 5;

a++;

Decrement --

- Decreases value by 1

```
let a = 5;
```

```
a--;
```

31. Conditional Statements

- Used to make decisions in code.
-

if

- Runs block if condition is true.

```
let age = 18;
```

```
if(age >= 18){  
  console.log("Adult");  
}
```

else

- Runs when condition is false.

```
let age = 16;
```

```
if(age >= 18){  
  console.log("Adult");  
}else{  
  console.log("Minor");  
}
```

else-if

- Checks multiple conditions.

```
let marks = 80;
```

```
if(marks >= 90){  
  console.log("A");  
}
```

```
}  
else if(marks >= 70){  
    console.log("B");  
}  
else{  
    console.log("C");  
}
```

switch

- Used for multiple fixed cases.

```
let day = 1;
```

```
switch(day){  
    case 1:  
        console.log("Monday");  
        break;  
  
    case 2:  
        console.log("Tuesday");  
        break;  
  
    default:  
        console.log("Invalid");  
}
```

- break stops further execution.
- default runs if no case matches.

32. Loops

- Used to repeat code.
-

for loop

- Used when number of iterations is known.


```
for(let i=1; i<=5; i++){  
  console.log(i);  
}
```

while loop

- Runs while condition is true.

```
let i = 1;
```

```
while(i <= 5){  
  console.log(i);  
  i++;  
}
```

do while

- Runs at least one time.

```
let i = 1;
```

```
do{  
  console.log(i);  
  i++;  
}while(i <= 5);
```

Nested loops

- Loop inside another loop.

```
for(let i=1; i<=2; i++){  
  for(let j=1; j<=2; j++){  
    console.log(i,j);  
  }  
}
```

break

- Stops loop completely.

```
for(let i=1; i<=5; i++){  
  if(i == 3){  
    break;  
  }  
  console.log(i);  
}
```

continue

- Skips current iteration.

```
for(let i=1; i<=5; i++){  
  if(i == 3){  
    continue;  
  }  
  console.log(i);  
}
```

33. Strings

- String = collection of characters.

```
let name = "Aditya";
```

String Methods

length

```
name.length
```

toUpperCase()

```
name.toUpperCase()
```

toLowerCase()

```
name.toLowerCase()
```

trim()

- Removes extra spaces.

```
" hello ".trim()
```

slice()

```
name.slice(0,3)
```

replace()

```
name.replace("A","R")
```

Template Literals

- Use backticks `
- Allows variable embedding.

```
let age = 20;
```

```
`My age is ${age}`
```

Escape Characters

Character Meaning

<code>\n</code>	New line
-----------------	----------

<code>\t</code>	Tab space
-----------------	-----------

<code>"</code>	Double quote
----------------	--------------

<code>'</code>	Single quote
----------------	--------------

<code>\</code>	Backslash
----------------	-----------

```
console.log("Hello\nWorld");
```

34. Arrays

- Array stores multiple values.

```
let arr = [1,2,3];
```

Array Methods

push()

- Add at end

```
arr.push(4);
```

pop()

- Remove from end

```
arr.pop();
```

shift()

- Remove from start

```
arr.shift();
```

unshift()

- Add at start

```
arr.unshift(0);
```

slice()

- Returns copy of portion

```
arr.slice(0,2);
```

splice()

- Add/remove elements

```
arr.splice(1,1);
```

forEach()

- Runs function for each element

```
arr.forEach((el)=>{  
  console.log(el);  
});
```

map()

- Creates new array

```
arr.map((el)=> el*2);
```

filter()

- Filters elements

```
arr.filter((el)=> el > 2);
```

reduce()

- Reduces array to single value

```
arr.reduce((res,el)=> res+el);
```

35. Objects

- Object stores data in key-value pairs.
-

Object Creation

```
let student = {  
  name: "Aditya",  
  age: 20  
};
```

Key-Value Pairs

```
let car = {  
  brand: "BMW",  
  color: "Black"  
};
```

- brand → key
- "BMW" → value

Access Values

```
car.brand  
car["color"]
```

Nested Objects

- Object inside object.

```
let student = {  
  name: "Aditya",  
  marks: {  
    math: 90,  
    science: 85  
  }  
};
```

Access

student.marks.math

Object Methods

- Functions inside objects.

```
let person = {  
  name: "Aditya",  
  
  greet(){  
    console.log("Hello");  
  }  
};
```

this keyword

- Refers to current object.

```
let person = {  
  name: "Aditya",  
  
  greet(){  
    console.log(this.name);  
  }  
};
```

- this.name → "Aditya"

36. Functions

- Functions are reusable blocks of code.
-

Function Declaration

```
function greet(){  
  console.log("Hello");  
}
```

Function Expression

```
const greet = function(){  
  console.log("Hello");  
};
```

Arrow Functions

```
const add = (a,b) => {  
  return a+b;  
};
```

Short Form

```
const add = (a,b) => a+b;
```

try...catch

- Used for error handling.
 - Prevents program crash.
-

Syntax

```
try{  
  
}  
catch(err){  
  
}
```

Example

```
try{  
  console.log(a);  
}  
catch(err){  
  console.log("Error");  
}
```

Error Object

```
try{
  console.log(a);
}
catch(err){
  console.log(err);
}
```

finally

- Always runs.

```
try{
  console.log("Hello");
}
catch(err){
  console.log(err);
}
finally{
  console.log("Done");
}
```

throw

- Creates custom error.

```
throw "Invalid";
```

Callback Functions

- Function passed as argument.

```
function greet(name){
  console.log(name);
}
```

```
function test(callback){
```



```
    callback("Aditya");  
  }  
  
test(greet);
```

Higher-Order Functions

- Function that takes/returns another function.

```
function multipleGreet(func){  
  func();  
}
```

Scope

Global Scope

```
let a = 10;
```

Function Scope

```
function test(){  
  let b = 20;  
}
```

Block Scope

```
{  
  let x = 5;  
}
```

- let and const are block scoped.
-

Hoisting

- Variables and functions move to top before execution.

Function Hoisting

```
greet();
```

```
function greet(){  
  console.log("Hello");  
}
```

}

Variable Hoisting

```
console.log(a);
```

```
var a = 10;
```

- var is hoisted.
- let and const are hoisted but in TDZ.

37. Advanced JavaScript Concepts

Execution Context

- Environment where JS code runs.

Types

- Global Execution Context
- Function Execution Context

```
let a = 10;
```

```
function test(){}

```

Call Stack

- Tracks function execution.
- Works on LIFO (Last In First Out).

```
function one(){
    two();
}
```

```
function two(){
  console.log("Hello");
}
```

```
one();
```

Lexical Scope

- Inner function can access outer function variables.

```
function outer(){  
  let a = 10;  
  
  function inner(){  
    console.log(a);  
  }  
  
  inner();  
}
```

Closures

- Function remembers outer variables even after outer function ends.

```
function outer(){  
  let count = 0;  
  
  return function(){  
    count++;  
    console.log(count);  
  };  
}
```

Hoisting

- Variables/functions move to top before execution.

```
console.log(a);
```

```
var a = 10;
```

- var → undefined
 - let/const → TDZ
-

Memory Allocation

- JS automatically allocates memory for variables and objects.

```
let a = 10;
```

Garbage Collection

- Removes unused memory automatically.

```
let a = {  
  name: "Aditya"  
};
```

```
a = null;
```

- Old object becomes removable from memory.

38. DOM Manipulation

DOM Tree

- DOM = Document Object Model
- HTML page converted into tree structure.

Document

```
└─ html  
  └─ body  
    └─ h1
```

Selecting Elements

getElementById()

```
document.getElementById("heading");
```

querySelector()

- Selects first matching element.

```
document.querySelector("h1");
```

```
document.querySelector(".box");
```

innerText

- Gets visible text.

```
h1.innerText = "Hello";
```

innerHTML

- Gets/sets HTML content.

```
div.innerHTML = "<b>Hello</b>";
```

textContent

- Gets all text content.

```
div.textContent
```

Changing Styles

```
h1.style.color = "red";
```

```
h1.style.backgroundColor = "yellow";
```

Creating Elements

```
let btn = document.createElement("button");
```

```
btn.innerText = "Click";
```

Add Element

```
document.body.append(btn);
```

Removing Elements

```
btn.remove();
```

39. Events

- Events are actions performed by user/browser.
-

onclick

- Runs function on click.

```
button.onclick = function(){
```

```
  console.log("Clicked");
```

```
};
```

this in Event Listener

- this refers to element on which event is attached.

```
button.addEventListener("click", function(){  
  console.log(this);  
});
```

- In arrow function, this does not refer to element.

```
button.addEventListener("click", ()=>{  
  console.log(this);  
});
```

Extracting Data

From Input

input.value

From Form

```
form.addEventListener("submit", (e)=>{  
  e.preventDefault();  
  
  console.log(input.value);  
});
```

From Event Target

```
button.addEventListener("click", (e)=>{  
  console.log(e.target);  
});
```

More Events

Event	Meaning
click	Mouse click
dblclick	Double click
mouseover	Mouse enter
mouseout	Mouse leave
keydown	Key press

Event	Meaning
keyup	Key release
input	Typing input
submit	Form submit
change	Value changed

addEventListener()

- Best way to handle events.

```
button.addEventListener("click", function(){
  console.log("Clicked");
});
```

Mouse Events

Event	Meaning
click	Mouse click
dblclick	Double click
mouseover	Mouse enters
mouseout	Mouse leaves

```
button.addEventListener("mouseover", ()=>{
  console.log("Hover");
});
```

Keyboard Events

Event	Meaning
keydown	Key pressed
keyup	Key released

```
input.addEventListener("keydown", ()=>{
  console.log("Key Pressed");
});
```

Form Events

Event Meaning

submit Form submitted

change Value changed

input Input typed

```
form.addEventListener("submit", ()=>{  
  console.log("Submitted");  
});
```

Event Object

- Stores event information.

```
button.addEventListener("click", (event)=>{  
  console.log(event);  
});
```

Common Properties

- event.target
- event.type
- event.key

40. Event Bubbling & Delegation

Event Flow

Capturing → Target → Bubbling

Capturing Phase

- Event moves from parent to child.

```
div.addEventListener("click", ()=>{  
  console.log("Div");  
}, true);
```

- true enables capturing.

Bubbling Phase

- Default behavior.
- Event moves from child to parent.

```
button.addEventListener("click", ()=>{  
  console.log("Button");  
});
```

stopPropagation()

- Stops event bubbling/capturing.

```
button.addEventListener("click", (e)=>{  
  e.stopPropagation();  
});
```

Event Delegation

- Add event listener to parent instead of multiple children.
- Uses event bubbling.

```
ul.addEventListener("click", (e)=>{  
  console.log(e.target);  
});
```

- Better performance
- Useful for dynamic elements

41. Browser APIs

setTimeout()

- Runs code once after delay.

```
setTimeout(()=>{  
  console.log("Hello");  
,2000);
```

- 2000 = 2 seconds

setInterval()

- Runs code repeatedly after interval.

```
setInterval(()=>{  
  console.log("Hi");  
},1000);
```

- 1000 = 1 second
-

clearTimeout()

```
let id = setTimeout(()=>{},2000);
```

```
clearTimeout(id);
```

clearInterval()

```
let id = setInterval(()=>{},1000);
```

```
clearInterval(id);
```

Local Storage

- Stores data permanently in browser.
- Data remains after refresh.

Store Data

```
localStorage.setItem("name","Aditya");
```

Get Data

```
localStorage.getItem("name");
```

Remove Data

```
localStorage.removeItem("name");
```

Session Storage

- Stores data temporarily.
- Data removed after tab/browser closes.

Store Data

```
sessionStorage.setItem("user", "Aditya");
```

Get Data

```
sessionStorage.getItem("user");
```

42. Object Oriented Programming (OOP)

Object

- Real-world entity with properties and methods.

```
let student = {  
  name: "Aditya"  
};
```

Classes

- Blueprint for creating objects.

```
class Student{  
  
}
```

Constructor

- Special method called automatically.

```
class Student{  
  constructor(name){  
    this.name = name;  
  }  
}
```

```
let s1 = new Student("Aditya");
```

Inheritance

- One class acquires properties of another class.

```
class Person{
```

```
greet(){  
  console.log("Hello");  
}  
}
```

```
class Student extends Person{  
  
}
```

Encapsulation

- Wrapping data and methods together.

```
class Bank{  
  balance = 1000;  
}
```

Prototypes

- Object from which other objects inherit properties.

```
let arr = [1,2,3];
```

```
arr.__proto__
```

43. ES6+ Features

Destructuring

- Extract values from array/object.

Array

```
let arr = [1,2];
```

```
let [a,b] = arr;
```

Object

```
let user = {
```

```
  name: "Aditya"  
};
```

```
let {name} = user;
```

Spread Operator ...

- Expands values.

```
let arr1 = [1,2];  
let arr2 = [...arr1,3];
```

Rest Operator ...

- Collects multiple values.

```
function sum(...nums){  
  console.log(nums);  
}
```

Default Parameters

- Default value if argument not passed.

```
function greet(name="User"){  
  console.log(name);  
}
```

Optional Chaining ?.

- Prevents errors for undefined values.

```
user.address?.city
```

Modules

- Split code into files.
-

export

```
export const name = "Aditya";
```

import

```
import {name} from "./app.js";
```

Call Stack

- Tracks function execution.
- Works on LIFO.

```
function one(){  
  two();  
}
```

```
function two(){  
  console.log("Hello");  
}
```

```
one();
```

```
one()  
two()  
console.log()
```

Breakpoint

- Used for debugging.
- Stops code execution at specific line.

Add in Browser DevTools

- Open Inspect
- Go to Sources
- Click line number

debugger

```
let a = 10;
```

```
debugger;
```

```
console.log(a);
```

- Execution pauses at debugger

44. Async JavaScript

Synchronous

- Code runs line by line.

```
console.log("A");  
console.log("B");
```

Asynchronous

- Some tasks run later.

```
setTimeout(()=>{  
  console.log("Hello");  
},2000);
```

Callback Hell

- Nested callbacks.
- Hard to read/manage.

```
setTimeout(()=>{  
  setTimeout(()=>{  
    console.log("Hello");  
  },1000);  
},1000);
```

Promises

- Handles async operations.

States

- Pending
- Resolved
- Rejected

```
let promise = new Promise((resolve,reject)=>{  
  resolve("Done");  
});
```

.then() and .catch()

promise

```
.then((res)=>{
  console.log(res);
})
.catch((err)=>{
  console.log(err);
});
```

Promise Chaining

promise

```
.then(()=>{
  return 10;
})
.then((res)=>{
  console.log(res);
});
```

async

- Makes function asynchronous.

```
async function hello(){

}
```

await

- Waits for promise result.

```
async function test(){
  let res = await promise;
  console.log(res);
}
```


CODE OF CALLBACK, PROMISES AND ASYNC/AWAIT

Callback Hell

```
h1 = document.querySelector("h1");
```

```
function changeColor(color, delay, nextColorChange) {
```

```
  setTimeout(() => {
```

```
    h1.style.color = color;
```

```
    if(nextColorChange){
```

```
      nextColorChange();
```

```
    }
```

```
  }, delay);
```

```
}
```

```
changeColor("red", 1000, () => {
```

```
  changeColor("orange", 1000, () => {
```

```
    changeColor("green", 1000, () => {
```

```
      changeColor("yellow", 1000, () => {
```

```
        changeColor("blue", 1000);
```

```
      });
```

```
    });
```

```
});
```

```
});
```

Promise Creation

```
function saveToDb(data){  
  
  return new Promise((resolve, reject) => {  
  
    let internetSpeed =  
      Math.floor(Math.random() * 10) + 1;  
  
    if(internetSpeed > 4){  
      resolve("success : data was saved");  
    } else {  
      reject("failure : weak connection");  
    }  
  
  });  
  
}
```

then() and catch()

```
saveToDb("apna college")  
  
.then(() => {  
  console.log("promise was resolved");  
})  
  
.catch(() => {
```

```
    console.log("promise was rejected");  
  });
```

Promise Chaining

```
saveToDb("apna college")
```

```
.then(() => {  
  console.log("data1 saved");  
  return saveToDb("helloworld");  
})
```

```
.then(() => {  
  console.log("data2 saved");  
  return saveToDb("shradha");  
})
```

```
.then(() => {  
  console.log("data3 saved");  
})
```

```
.catch(() => {  
  console.log("promise was rejected");  
});
```

Promise with Result & Error

```
saveToDb("apna college")
```

```
.then((result) => {  
  console.log("data1 saved");  
  console.log(result);  
})
```

```
    return saveToDb("helloworld");  
  })
```

```
.then((result) => {  
  console.log("data2 saved");  
  console.log(result);  
})
```

```
.catch((error) => {  
  console.log("promise rejected");  
  console.log(error);  
});
```

Promise Example

```
function changeColor(color, delay){  
  
  return new Promise((resolve, reject) => {  
  
    setTimeout(() => {  
  
      h1.style.color = color;  
      resolve("color changed");  
  
    }, delay);  
  
  });  
  
}
```

Promise Chaining with Colors

```
changeColor("red",1000)
```

```
.then(() => {  
  console.log("red completed");  
  return changeColor("orange",1000);  
})
```

```
.then(() => {  
  console.log("orange completed");  
  return changeColor("green",1000);  
})
```

```
.then(() => {  
  console.log("green completed");  
  return changeColor("blue",1000);  
})
```

```
.then(() => {  
  console.log("blue completed");  
});
```

Promise Reject Example

```
function changeColor(color, delay){  
  
  return new Promise((resolve, reject) => {  
  
    setTimeout(() => {  
  
      let num =  
        Math.floor(Math.random() * 5) + 1;  
  
      if(num > 3){
```

```
        reject("promise rejected");
    }

    h1.style.color = color;
    resolve("color changed");

    }, delay);

});

}
```

async Function

```
async function greet(){
    return "hello!";
}
```

async with Error

```
async function greet(){

    throw "some random error";

    return "hello!";
}
```

async then catch

```
greet()

.then((result) => {
    console.log("promise resolved");
    console.log(result);
})
```

```
}}
```

```
.catch((err) => {  
  console.log("promise rejected");  
  console.log(err);  
});
```

await

```
async function demo(){  
  
  await changeColor("red",1000);  
  
  await changeColor("orange",1000);  
  
  await changeColor("green",1000);  
  
  changeColor("blue",1000);  
  
}
```

try catch with await

```
async function demo(){  
  
  try{  
  
    await changeColor("red",1000);  
  
    await changeColor("orange",1000);  
  
    await changeColor("green",1000);
```

```
    await changeColor("blue",1000);

}

catch(err){

    console.log("error caught");
    console.log(err);

}

}
```

await Example

```
function getNum(){

    return new Promise((resolve, reject) => {

        setTimeout(() => {

            let num =

                Math.floor(Math.random() * 10) + 1;

            console.log(num);

            resolve();

        },1000);

    });

}
```



```
}
```

```
async function demo(){
```

```
  await getNum();
```

```
  await getNum();
```

```
  await getNum();
```

```
  await getNum();
```

```
}
```